SimSpect

Automated litmus testing for simulated hardware Spectre defenses

Anna Eaton, Caroline Trippel

Hardware side-channel attacks (HSCAs): Attackers exploit data-dependent hardware resource usage to infer operand value, under some execution circumstance.

Architecturally dete
execution path passe
unsafe instruction

Constant-time code **(SW)**

Transient, microarchitecturally determined execution path passes data to unsafe instruction

Meltdown atta
privileged data
permissions exc
handling

Kernel page table isolation **(KPTI)**

Spectre attacks use architecturally legal instructions + incorrect speculative control flow

STT

SPT

Spec Shield

NDA

DoM

SDO

Doppel-ganger loads

Protean

Recon

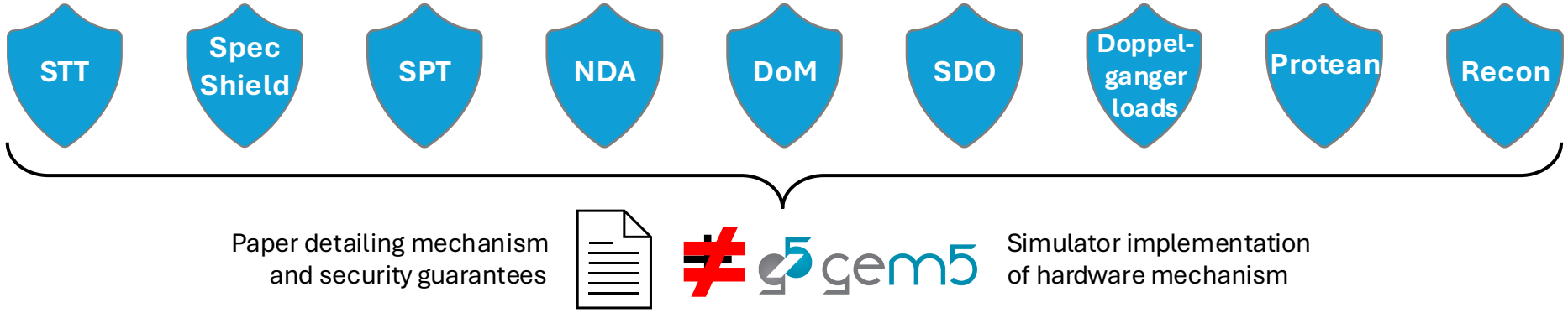
- All memory
- All registers
- Architecturally untransmitted memory
- Annotated pages and dependent registers
- …

×

- Load leaks ad s
- Branch leaks branch condition
- ALUop leaks operand
- Store leaks address
- …

×

- Store-to-load forwarding address prediction
- Branch address prediction
- Branch target prediction
- Load value prediction
- …



- **Many HW-modifying Spectre defenses** — with different mechanisms and security guarantees
- **PPA essential** in HW modifications, simulator performance **invalid** if it doesn't uphold security guarantees
  - Frequent bugs in security contract of implementation, need faster iteration

Given an **arbitrary proposed hardware Spectre defense**, with a gem5 architectural simulator proof of concept, can we

1. **formalize the security properties** that the defense will uphold
2. **rigorously validate** them on the implementation?

We want a **comprehensive** set of tests that covers the space of leakage that a particular defense should protect against:

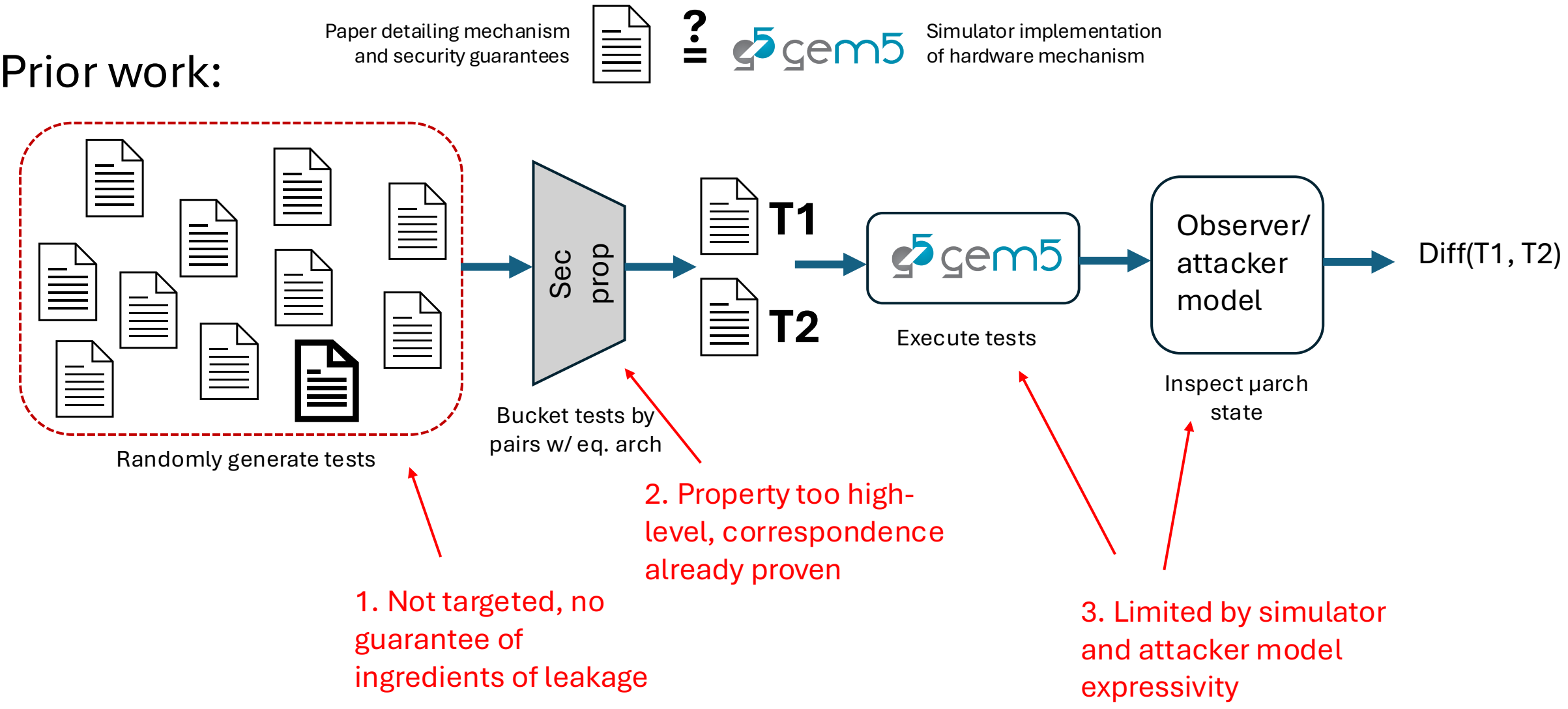
1. Defense-specific
2. Targeted to cause leakage

Prior work

SPEC2017 Benchmark suite	• Tests to an incorrect security contract • Tests do not target leakage
Spectre benchmark suite	• Tests to an incorrect security contract • Tests target leakage
Handmade defense- specific litmus tests	• Tests are not comprehensive • Tests target leakage
AMuLeT (security contract targeted fuzzing)	• Tests to an overly broad security contract • Tests do not target leakage (random)

[AMuLeT: Fu et. al, ASPLOS '25]

Prior work:



[AMuLeT: Fu et. al, ASPLOS '25]

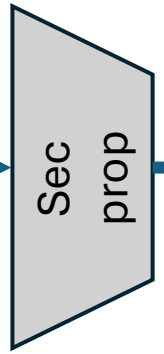
Paper detailing mechanism and security guarantees

?

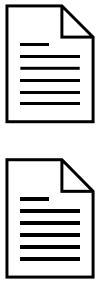
Simulator implementation of hardware mechanism

Prior work:

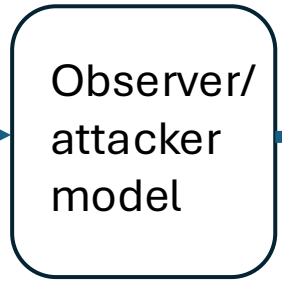
Randomly generate tests



Bucket tests by pairs w/ eq. arch



Execute tests

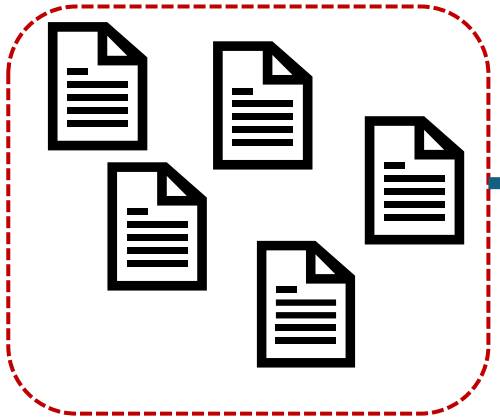


Inspect μarch state

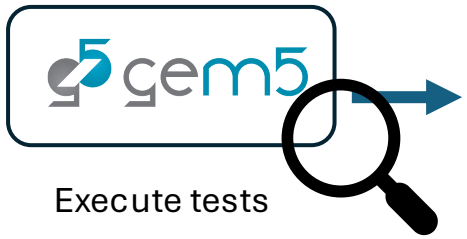
Diff(T1, T2)?

SimSpect:

Precise defense security guarantee



Fewer, shorter, more targeted tests



Execute tests

Did the test execution occur?

Get pipeline output

[AMuLeT: Fu et. al, ASPLOS '25]

Formal language specification

Each defense has:

1. **Protection set** - the set of architectural state* that the defense promises to not leak
2. **Leakage contract** - the set of pairs of transmitters and their unsafe operand(s)
3. **Execution contract** - what speculation primitives the defense considers

* = plus **protection set propagation rule -** how the protection set changes throughout time with different instructions committing

Hardware side-channel attacks (HSCAs): Attackers exploit data-dependent hardware resource usage to infer operand value, under some execution circumstance.

- All memory
- All registers
- Architecturally untransmitted memory
- Annotated pages and dependent registers
- …

×

- Load leaks address
- Branch leaks branch condition
- ALUop leaks operand
- Store leaks address
- …

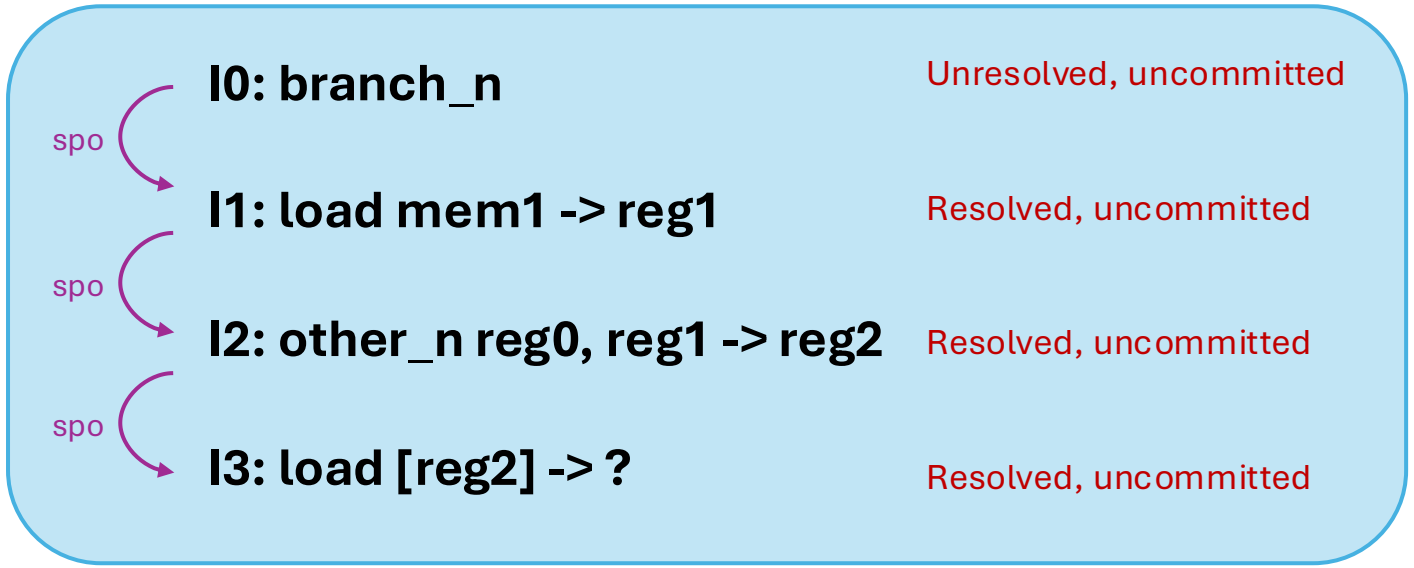
×

- Store-to-load forwarding address prediction
- Branch address prediction
- Branch target prediction
- Load value prediction
- …

We want to make *test executions*

- Test programs, *at some moment in time*, annotated with microarchitectural state

- Series of abstract instructions, connected in a total order by spo (speculative program order) edges
- With state (registers, memory) objects semispecified for the operands
- With **resolved** and **committed** annotations



Comprehensive test generation

***Alloy* axiomatic model finder, that has two components**

1. rules for what an execution is

(consistent across all defenses)

The "test" we are trying to generate

Model

Nodes:
- Instructions:
  - Xmit
    - Ld
    - Str
    - Br
    - other
  - Non-xmit
    - Br
    - other
- State: reg, mem

Booleans:
- Committed (out of ROB, non-spec)
  - Contiguous at end of SPO
- Resolved
  - Br knows target and condition
  - Ld str know addr

Edges:
- Ordering
  - Po: i->i
  - Spo: i->i
  - Rf_r, rf_m, co_r, co_m, fr_r, fr_m: i->i
- Labeling
  - In_addr, in_reg, in_mem, out_reg, out_mem: i->state
  - Bool: i->bool

2. rules for what defines a *violating* execution *(defense-specific, comes from the security contract that we just specified), in the syntax of (1)*

STT

SimSpect specification

- **Protection set:** Mem (- memory used in committed loads)
- **Leakage contract:** ld->in_addr, $xmit_{other}$->in_xmit, br->in_reg
- **Execution contract:** Architect-defined visibility point, definition of speculative:
  - i not committed
  - ∄ nonresolved i.(~po).Br

Alloy snippets - STT

```
fun execution_contract_p[p: PTag->univ] : Instruction {uncommitted_p[p] & has_unresolved_brs_p[p]}
fun protection_set: State {Mem_s}
fun leakage_contract : Operand {Loads.inaddr+(Branchxs+Otherxs).inreg}
fun prot_set_propagation_p[p:PTag->univ,i:Instruction,s:State] : State {
        s - (Loads & no_unresolved_brs_p[p] & i).inmem.opstate
}
```

STT security specification

```
fun speculative_xmit_p[p: PTag->univ] : Operand {leakage_contract_p[p] <: (execution_contract_p[p].operands)}

pred secure_speculation_scheme_p[p: PTag->univ] {
    (no (last_committed_protset_p[p].(~opstate) & speculative_xmit_p[p])) and
    (no last_committed_protset_p[p].(~opstate).(^(op_edges_p[p])) & speculative_xmit_p[p])
}
```

Definition of secure speculation scheme
(when an instance is not a **violation**)

```
let gen_useful_litmus {
  not secure_speculation_scheme_p[no_p]
}

run gen_lit {
  gen_useful_litmus
} for 6 but exactly 4 Instruction, exactly 1 rBool, exactly 1 cBool, exactly 1 tBool
```

Generate all possible violating
tests up to a test-length bound!

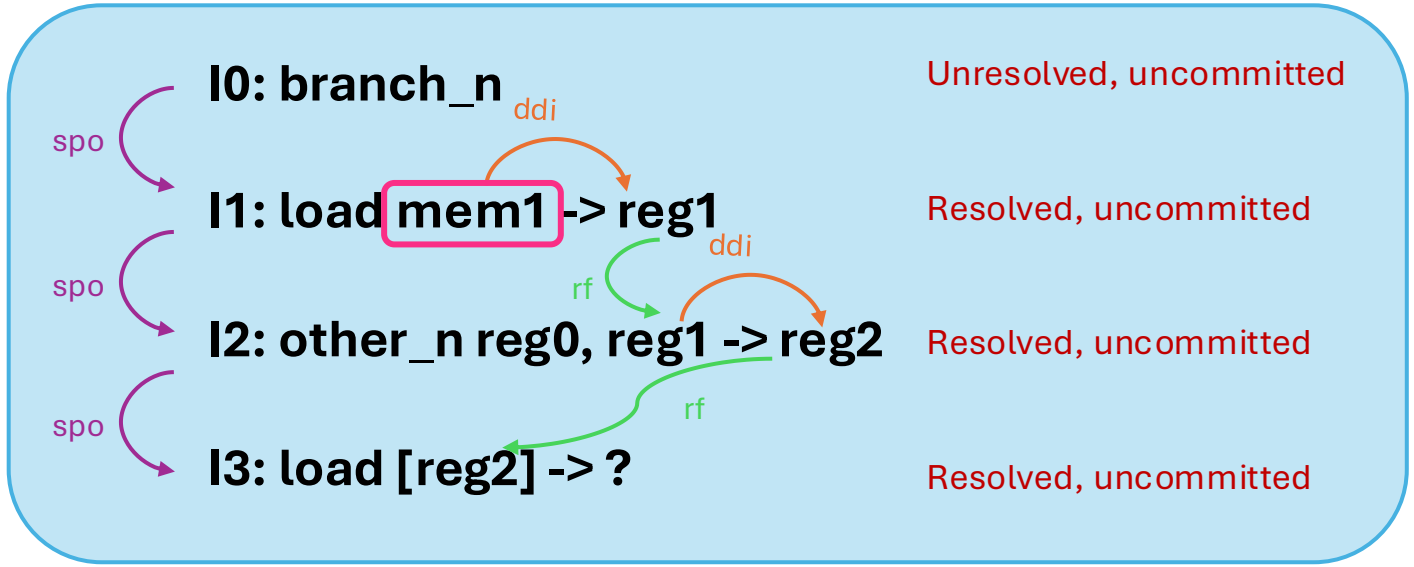
We want to make *test executions*

- Test programs, *at some moment in time*, annotated with microarchitectural state

STT

SimSpect specification

- **Protection set:** Mem (- memory used in committed loads)
- **Leakage contract:** ld->in_addr, $xmit_{other}$->in_xmit, br->in_reg
- **Execution contract:** Architect-defined visibility point, definition of speculative:
  - i not committed
  - ∄ nonresolved i.(~po).Br



Last_committed_protset is always **mem** because no committed instructions

Rf: reads from
Ddi: data-dependence internal

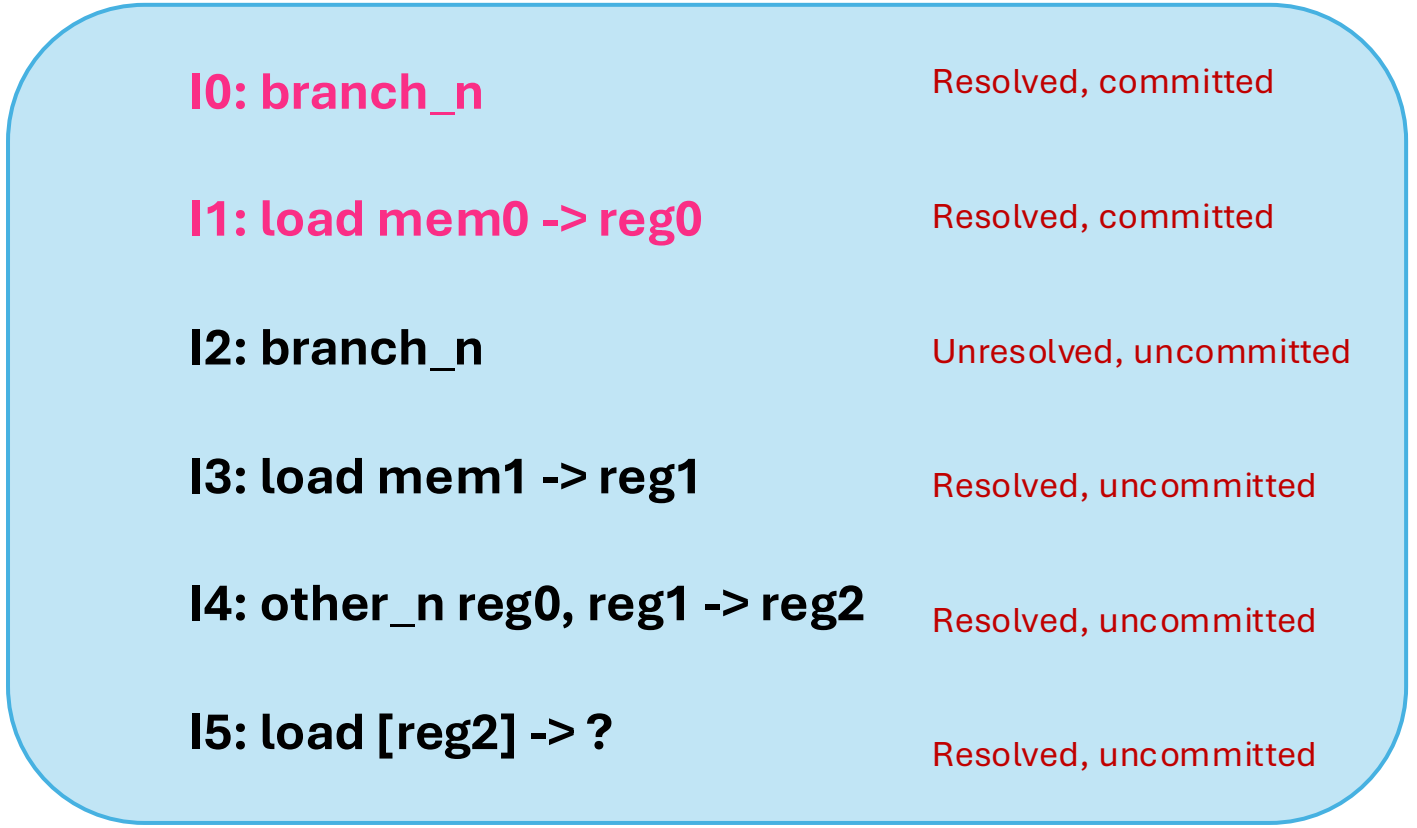
Together they form data dependence edges

Abstract execution generated, now what??

- Concretization phase
  1. **Interleave instructions** – insert extra instructions, has effects on timing and various windows (Alloy, python)
  2. **Specify state** - go through and concretize the states from the reserved register pool (caller-saved registers)
  3. **Specify instructions** – put LLVM instructions in
  4. **Speculation primitive annotations/specification** – concretize branch inputs and track annotations
  5. **Specify remaining state** – the state that wasn't specified in alloy, fill from the untouched registers
  6. **Build LLVM** – force the registers and make mem instrs volatile, add prelude for clearing mem array, flushing predicates to lengthen branch resolutions

Pink means new addition as of that phase

1. **Interleave instructions** – insert extra instructions, has effects on timing and various windows (Alloy, python)



2. Specify state - go through and concretize the states from the reserved register pool (caller-saved registers)

Program *(abstract instructions, specified state pinned)*

```
pc0:  br_x    inreg0=Reg_s$0, ? c, r
pc1:  ld      inmem=Mem_s$0, outreg=Reg_s$0, ? c, r
pc2:  br_x    inreg0=Reg_s$0, ?
pc3:  ld      inmem=Mem_s$0, outreg=Reg_s$1, ?
pc4:  other_n inreg0=Reg_s$0, inreg1=Reg_s$1, outreg=Reg_s$0
pc5:  ld      inaddr=Reg_s$0, inmem=Mem_s$0, outreg=Reg_s$0
```

Annotations

Resources: 2 registers (Reg_s$0, Reg_s$1), 1 memory (Mem_s$0)

Xm: pc5

3. **Specify instructions** – put LLVM instructions in

Program *(concrete instructions)*

```
pc0:  br_cond   (br i1)          inreg0=Reg_s$0 c, r
pc1:  load      (load volatile)  inmem=Mem_s$0 → outreg=Reg_s$0 c, r
pc2:  br_cond   (br i1)          inreg0=Reg_s$0
pc3:  load      (load volatile)  inmem=Mem_s$0 → outreg=Reg_s$1
pc4:  add       (add i64)        inreg0=Reg_s$0, inreg1=Reg_s$1 → outreg=Reg_s$0
pc5:  load      (load volatile)  inaddr=Reg_s$0, inmem=Mem_s$0 → outreg=Reg_s$0
```

Annotations

Resources: 2 registers (Reg_s$0, Reg_s$1), 1 memory (Mem_s$0)

Xm: pc5

- Takes input from instruction_table jsons that map abstract instructions to concrete instructions
- # of specified operands has to fit within specific instruction shape

4. **Speculation primitive annotations/specification** – concretize branch inputs and track annotations

Program *(with branch directions)*

```
pc0:  br_cond  [correctly_not_taken → bb_1]    inreg0=Reg_s$0 c, r
pc1:  load     inmem=Mem_s$0 → outreg=Reg_s$0 c, r
pc2:  br_cond  [mispredict_not_taken → bb_3]   inreg0=Reg_s$0
pc3:  load     inmem=Mem_s$0 → outreg=Reg_s$1
pc4:  add      inreg0=Reg_s$0, inreg1=Reg_s$1 → outreg=Reg_s$0
pc5:  load     inaddr=Reg_s$0, inmem=Mem_s$0 → outreg=Reg_s$0
```

Annotations

Resources: 2 registers (Reg_s$0, Reg_s$1), 1 memory (Mem_s$0)

Xm: pc5

+ branch annotation table:

	pc0 (branch_outer)	pc2 (branch_inner)
mode	correctly_not_taken	mispredict_not_taken
condition_value	false (not taken)	true (taken)
taken_target	end_block	end_block
fallthrough_target	bb_1	bb_3
btb_prediction	fall_through	fall_through
btb_predicted_pc	1	3

	taken	**Not taken**
mispred		
pred		

5. Specify remaining state – the state that wasn't specified in alloy, fill from the untouched registers

Program (physical x86 registers assigned):

```
pc0:  br_cond  rax              [correctly_not_taken → bb_1]
pc1:  load     [mem+0] → rax    (inaddr=r9, for base pointer)
pc2:  br_cond  rax              [mispredict_not_taken → bb_3]
pc3:  load     [mem+0] → rcx    (inaddr=r9)
pc4:  add      rax, rcx → rax   (leaq)
pc5:  load     [probe+rax] → rax (inaddr=rax, inmem offset=0)
```

Annotations

Xm: pc5

	pc0 (branch_outer)	pc2 (branch_inner)
mode	correctly_not_taken	mispredict_not_taken
condition_value	false (not taken)	true (taken)
taken_target	end_block	end_block
fallthrough_target	bb_1	bb_3
btb_prediction	fall_through	fall_through
btb_predicted_pc	1	3

Register mapping	
Reg_s$0 → rax	locked
Reg_s$1 → rcx	locked
Mem_s$0 → offset 0	1 slot, 8 bytes
virtual pool	rdx, rsi, rdi, r8, r9, r10, r11
condition_assigned	pc0: false, pc2: true

6. Build LLVM – force the registers and make mem instrs volatile, add prelude for clearing mem array, flushing predicates to lengthen branch resolutions (abbreviated)

```
entry:
  ; zero-init and flush all condition slots
  clflush (cond_slot_pc0), clflush (cond_slot_pc2)

  ; pc=0  branch_outer (br_cond)
  movq (cond_slot_pc0), %rsi
  icmp ne → br i1 %cond, label %end_block, label %bb_1
                                ;

bb_1:
  ; pc=1  load_A → Ra (load)
  movq 0(%mem_base), %rax          ; Ra = rax_1  last committed

  ; pc=2  branch_inner (br_cond) — unresolved, mispredicted  first noncommitted
  movq (cond_slot_pc2), %rsi
  icmp ne → br i1 %cond, label %end_block, label %bb_3

bb_3:
  ; pc=3  load_B → Rb (load)
  movq 0(%mem_base), %rcx          ; Rb = rcx_1

  ; pc=4  add Ra, Rb → Rc
  leaq (%rax, %rcx), %rax          ; Rc = rax_2 = Ra + Rb

  ; pc=5  load[Rc] → Rd (load)
  movq (%probe_mem, %rax, 1), %rax ; Rd = rax_3 = probe_mem[Rc] xm

  br label %end_block
end_block:
  ret i64 0
```

Entry	Value
xmit	pc=5, kind=ld, x86_offset=0xc8e
last_committed	pc=1 (load_A), x86_offset=0xc6e
first_noncommitted	pc=2 (branch_inner), x86_offset=0xc77
branch pc0	correctly_not_taken, x86 branch at 0xc64 (jne), fallthrough=0xc66
branch pc2	mispredict_not_taken, x86 branch at 0xc7d (jne), BTB predicted=0xc7f, actual target=0x401cb2

7. Compile to X86 (abbreviated)

```
; --- setup: flush condition slots from cache ---
    mfence
    clflush (%rdx)          ; flush cond_slot_pc0
    clflush (%rax)          ; flush cond_slot_pc2
    clflush (%rcx)          ; flush cond_ptr_slot_pc2
    mfence

    ; --- pc0 [0xc5e]: branch_outer (resolved, committed, correctly not-taken) ---
    ;     branch insn at [0xc64]
__litmus_..._pc0:                       ; 0xc5e
    movq    (%rdx), %rsi                  ; slow load condition (cache miss)
    testq   %rsi, %rsi
    jne     .LBB0_3                     ; 0xc64 — not taken (condition is 0) → falls through

    ; --- pc1 [0xc6e]: load_A → Ra  [LAST COMMITTED] ---
__litmus_..._pc1:                       ; 0xc6e
__litmus_..._last_committed:
    movq    (%rdi), %rax                  ; Ra = mem[0] → rax

    ; --- pc2 [0xc77]: branch_inner (unresolved, mispredicted)  [FIRST NONCOMMITTED] ---
    ;     branch insn at [0xc7d], BTB predicted fall-through to [0xc7f]
__litmus_..._pc2:                       ; 0xc77
__litmus_..._first_noncommitted:
    movq    (%rdx), %rsi                  ; slow load condition (cache miss → big window)
    testq   %rsi, %rsi
    jne     .LBB0_3                     ; 0xc7d — TAKEN to end_block (0x401cb2)
                                        ; BTB predicts not-taken → speculative fall-through ↓

    ; --- pc3 [0xc7f]: load_B → Rb  (speculative) ---
__litmus_..._pc3:                       ; 0xc7f
    movq    (%rdi), %rcx                  ; Rb = mem[0] → rcx

    ; --- pc4 [0xc85]: add Ra, Rb → Rc  (speculative) ---
__litmus_..._pc4:                       ; 0xc85
    leaq    (%rax,%rcx), %rax             ; Rc = Ra + Rb → rax

    ; --- pc5 [0xc8e]: load[Rc] → Rd  ★XMIT  (speculative) ---
__litmus_..._pc5:                       ; 0xc8e
    movq    (%rcx,%rax), %rax             ; probe_mem[Rc] → THE LEAK

.LBB0_3:                                ; end_block
    xorl    %eax, %eax
    retq
```

Entry	Value
xmit	pc=5, kind=ld, x86_offset=0xc8e
last_committed	pc=1 (load_A), x86_offset=0xc6e
first_noncommitted	pc=2 (branch_inner), x86_offset=0xc77
branch pc0	correctly_not_taken, x86 branch at 0xc64 (jne), fallthrough=0xc66
branch pc2	mispredict_not_taken, x86 branch at 0xc7d (jne), BTB predicted=0xc7f, actual target=0x401cb2

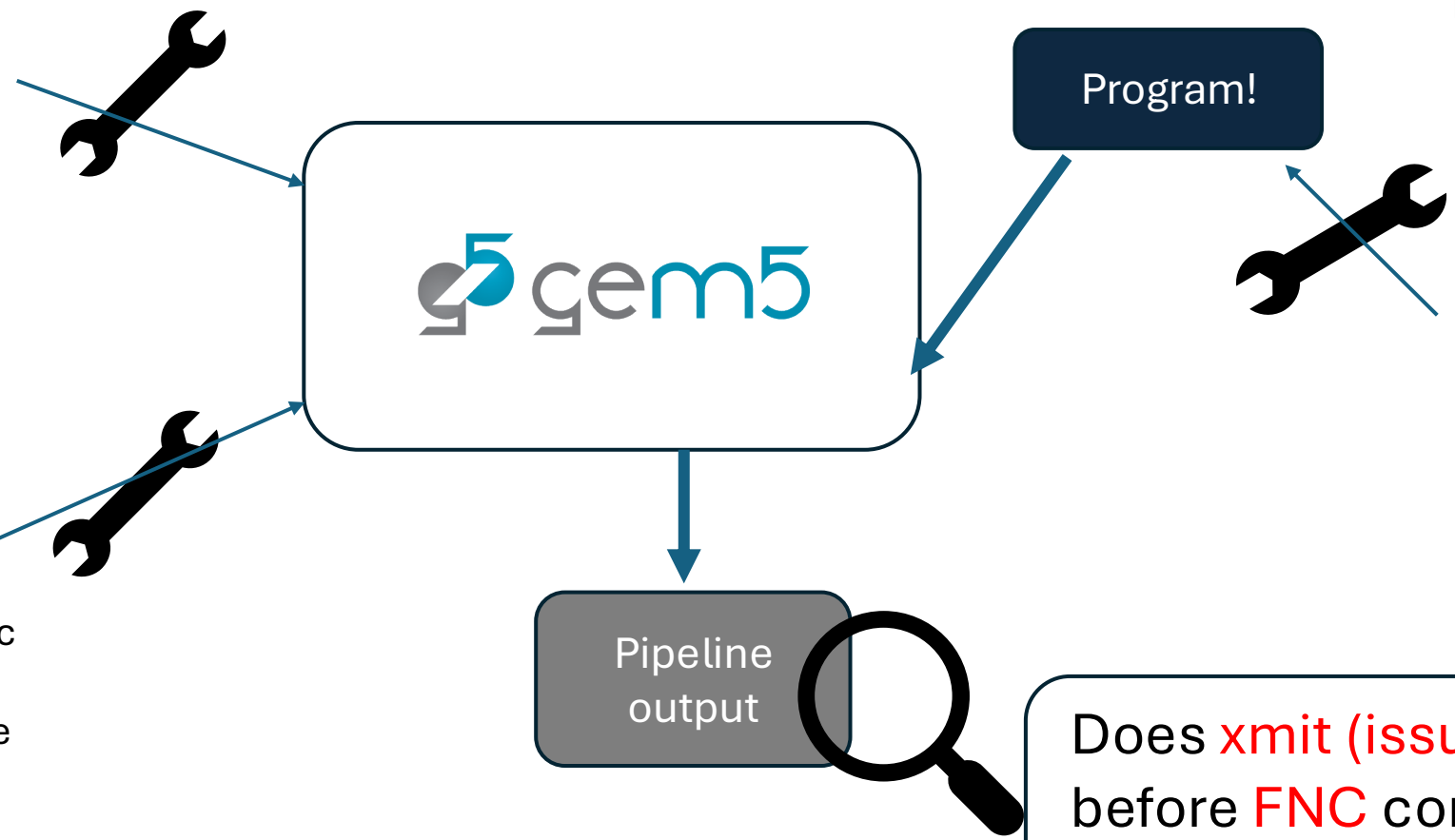
Running the tests!!

Entry	Value
xmit	pc=5, kind=ld, x86_offset=0xc8e
last_committed	pc=1 (load_A), x86_offset=0xc6e
first_noncommitted	pc=2 (branch_inner), x86_offset=0xc77
branch pc0	correctly_not_taken, x86 branch at 0xc64 (jne), fallthrough=0xc66
branch pc2	mispredict_not_taken, x86 branch at 0xc7d (jne), BTB predicted=0xc7f, actual target=0x401cb2

Explicit tuning (in simulator)

Necessary:
- force speculation to match annotations
OVERRIDE BTB WITH TARGET

Unnecessary but nice:
- Lengthen commit/noncommit window by stalling fnc
- Stall branch resolution to increase window



Implicit tuning (in program)

Unnecessary but nice:
- Lengthen commit/noncommit window by inserting nops
- Flush branch condition to increase speculation window length

Does xmit (issue or complete) before FNC commits and after LC commits?

Inspection of abbreviated pipeline

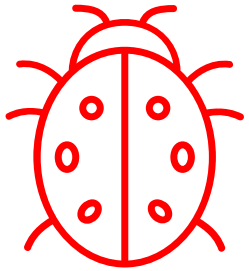
```
// f = fetch, d = decode, n = rename, p = dispatch, i = issue, c = complete, r = retire

                                  timeline                                                        tick      pc.upc       disasm                 seq_num     timestamps
[.....................................icr.....................................f.dnp...............]-(  2520000) 0x00401c78.1 CLFLUSH_M.mfence   [   556]  f,2671000, r,2777500
[......................................icr....................................f.dnp...............]-(  2520000) 0x00401c7b.0 MFENCE             [   557]  f,2671000, r,2780500
[...........fdnp.........................i..........cr............................................]-(  2700000) 0x00401c7e.0 MOV_R_M            [   558]  f,2722000, r,2803000
[...........fdnp...................................icr............................................]-(  2700000) 0x00401c81.0 TEST_R_R           [   559]  f,2722000, r,2803500
[...........fdnic...................................r.............................................]-(  2700000) 0x00401c84.0 JNZ_I              [   560]  f,2722000, r,2803500
[...........fdnic...................................r.............................................]-(  2700000) 0x00401c84.1 JNZ_I              [   561]  f,2722000, r,2803500
[...........fdnp...................................icr............................................]-(  2700000) 0x00401c84.2 JNZ_I              [   562]  f,2722000, r,2804000
[...........fdnic....................................r............................................]-(  2700000) 0x00401c86.0 LEA_R_M            [   563]  f,2722000, r,2804000
[...........fdnp........................i............cr...........................................]-(  2700000) 0x00401c8e.0 MOV_R_M            [   564]  f,2722000, r,2807000
[...........fdnp...................................ic...r..........................................]-(  2700000) 0x00401c91.0 MOV_R_R            [   565]  f,2722000, r,2807000
[...........fdnp...................................i...........cr.................................]-(  2700000) 0x00401c94.0 MOV_R_M            [   566]  f,2722500, r,2826000
[...........fdnp..............................................i...........cr.....................]-(  2700000) 0x00401c97.0 MOV_R_M            [   567]  f,2722500, r,2849000
[...........fdnp.........................................................ic.r.....................]-(  2700000) 0x00401c9a.0 TEST_R_R           [   568]  f,2722500, r,2850000
[...........fdnic.........................................................r......................]-(  2700000) 0x00401c9d.0 JNZ_I              [   569]  f,2722500, r,2850000
[...........fdnic.........................................................r......................]-(  2700000) 0x00401c9d.1 JNZ_I              [   570]  f,2722500, r,2850000
[...........fdn.p........................................................icr.....................]-(  2700000) 0x00401c9d.2 JNZ_I              [   571]  f,2722500, r,2850000
[,,,,,,,,,,,fdn,p,,,,,,,,,,,,,,,,,,,,,,,,i,,,,,,,,,,,,,c,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,]-(  2700000) 0x00401c9f.0 -----MOV_R_M       [   572]  f,2722500, r,0
[,,,,,,,,,,,fdn,p,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,ic,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,]-(  2700000) 0x00401ca2.0 -----LEA_R_M       [   573]  f,2722500, r,0
[,,,,,,,,,,,,f,dnp,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,ic,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,]-(  2700000) 0x00401ca6.0 -----LEA_R_M       [   574]  f,2723000, r,0
[,,,,,,,,,,,,f,dnp,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,ic,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,]-(  2700000) 0x00401cae.0 -----MOV_R_M       [   575]  f,2723000, r,0
[,,,,,,,,,,,,f,dnp,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,ic,,,,,,,,,,,,,,,,,,,]-(  2700000) 0x00401cb2.0 -----XOR_R_R       [   576]  f,2723000, r,0
[,,,,,,,,,,,,f,dnic,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,]-(  2700000) 0x00401cb4.0 -----MOV_R_R       [   577]  f,2723000, r,0
```

FNC/LC window

Xmit occurred between last_committed commit and first_noncommitted commit ☑

Clarifications

- Transmitter detection
  - Issue vs complete vs don’t squash before outer squash
  - Need transmitter+defense specific criterion



???

STT Alloy
- Instruction sizes 3, 4, 5, 6

STT implementation

I0: branch_n	Resolved, committed
I1: load mem0 -> reg0	Resolved, committed
I2: branch_n	Unresolved, uncommitted
I3: load mem1 -> reg1	Resolved, uncommitted
I4: other_n reg0, reg1 -> reg2	Resolved, uncommitted
I5: load [reg2] -> ?	Resolved, uncommitted

Explanation:

- Unprotects memory when oldest tainted load becomes nonspeculative
- it should be the youngest tainted load

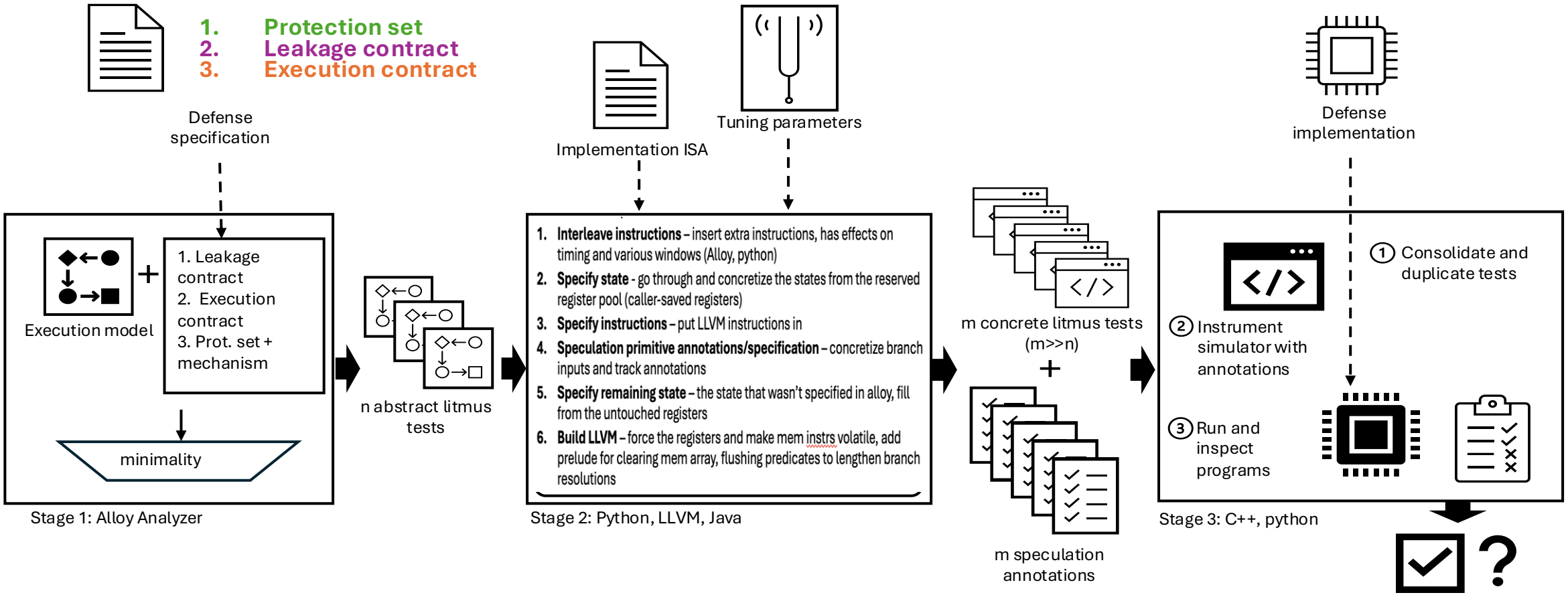
Preliminary results

- STT closed-source implementation (not from original authors):
  - Load ordering bug
    - We were aware of this bug's existence, but able to recreate it through our Alloy generation process
  - No recognition of any ALU-op transmitters
    - None of our other_x category, which is ambiguous in the STT paper, left up to the user to define all xmits, but in these defense implementations no other_x
  - Branch stall time
    - Only prevents prediction or resolution leakage, doesn't necessarily stall bug, so to see branch leakage you have to observe a younger mispredict on the dirty data squash before the bigger speculation squashes
- SPT
  - Working on running set on SPT
  - Tuning windows so that protection set tracking can be well-studied
- Near future:
  - Run bigger sets on AMuLeT eval defenses
  - Categorize AMuLeT random defenses by SimSpect skeleton (if present)

Discussion

- We motivate our test generation by potential leakage, front-loading the question of what is leakage to the test generation
- Separating out attack model problem, transmitter discovery problem, security-guarantee-to-high-level-security-property proof
- Customizable infrastructure such that you can test limited by gem5 possibilities (speculation primitives, transmitters) or test more robust properties on a simulator that might not actually exhibit the timing differentiation or whatever you're targeting
- We leverage the white-box nature of the simulator, but one can also decouple the front end of test-gen and do regular relational testing

SimSpect zoomed-out pipeline:



QUESTIONS?